אוניברסיטת בן-גוריון בנגב
Ben-Gurion University of the Negev

Applied Deep Learning

Omri Azencot

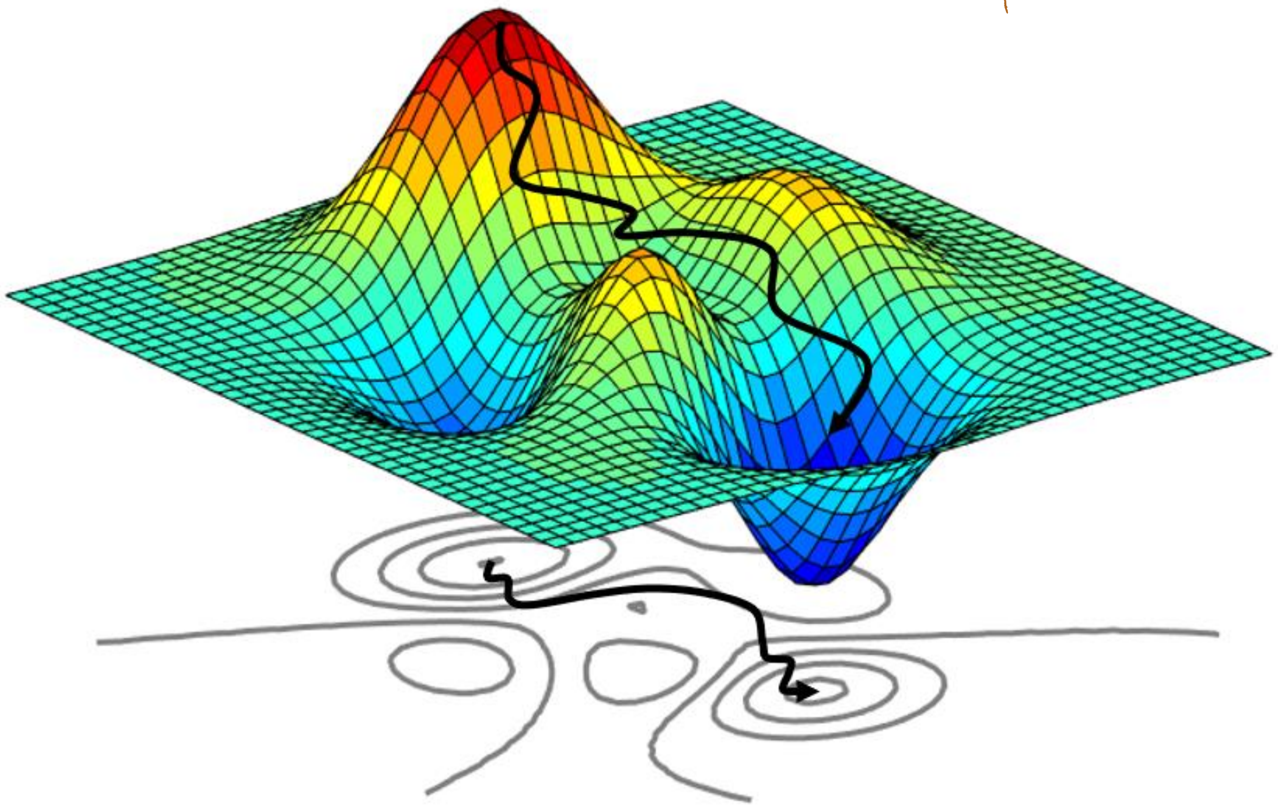
The Stein Faculty of Computer and Information Science

Ben-Gurion University of the Negev

Numerical Computation

Numerical Computation

loss function



$\theta = (x, y)^T$

parameters

Overflow and underflow

Challenge: represent infinite many real numbers with finite memory

rounding error

Underflow: near zero numbers are rounded to zero

For instance, avoid division by 0

Overflow and underflow

Challenge: represent infinite many real numbers with finite memory

Overflow: large numbers are approximated as $\pm\infty$

Further arithmetic will change
those into not-a-number values

Example: softmax function

The softmax function

$$\mathrm{softmax}(x)_i = \frac{\exp(x_i)}{\sum_{j=1}^{n} \exp(x_j)} \longleftarrow \text{all } x_i = c$$

If c is very negative => underflow

If c is very large => overflow

How to stabilize softmax?

Poor conditioning

How small changes in inputs affect function values

For instance, $f(x) = A^{-1}x$, the condition number is

$$\max_{i,j} \left| \frac{\lambda_i}{\lambda_j} \right|$$

Gradient-based optimization

Optimization refers to minimizing $f(x)$ by altering x

$$x^* = \arg\min_x f(x)$$

How can this be done?

Gradient-based optimization

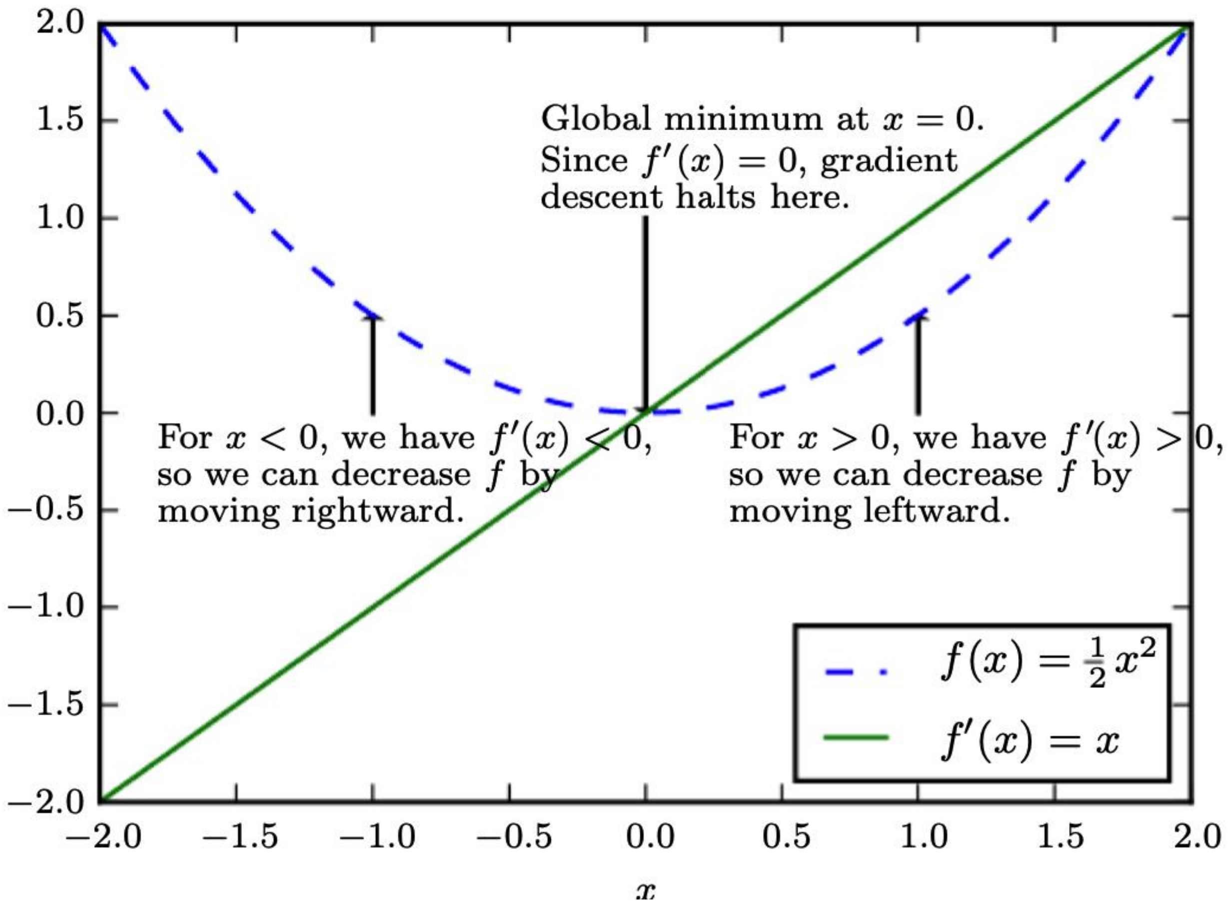
Let $y = f(x)$ be a function. Its derivative reads

$$f(x+\epsilon) \approx f(x) + \epsilon f'(x)$$

where $f'(x) = dy/dx$

For instance, $f\left(x - \epsilon \operatorname{sign}\left(f'(x)\right)\right) < f(x)$ $\longleftarrow$ gradient descent

Gradient descent



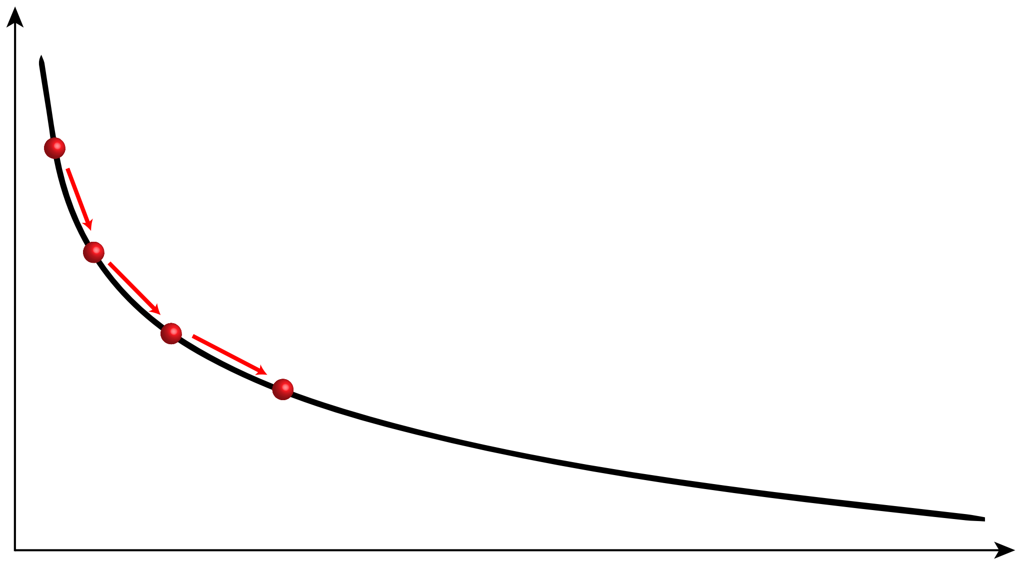
Gradient descent

We know

$$f(x + \mathrm{d}x) \approx f(x) + \mathrm{d}x \cdot f'(x)$$

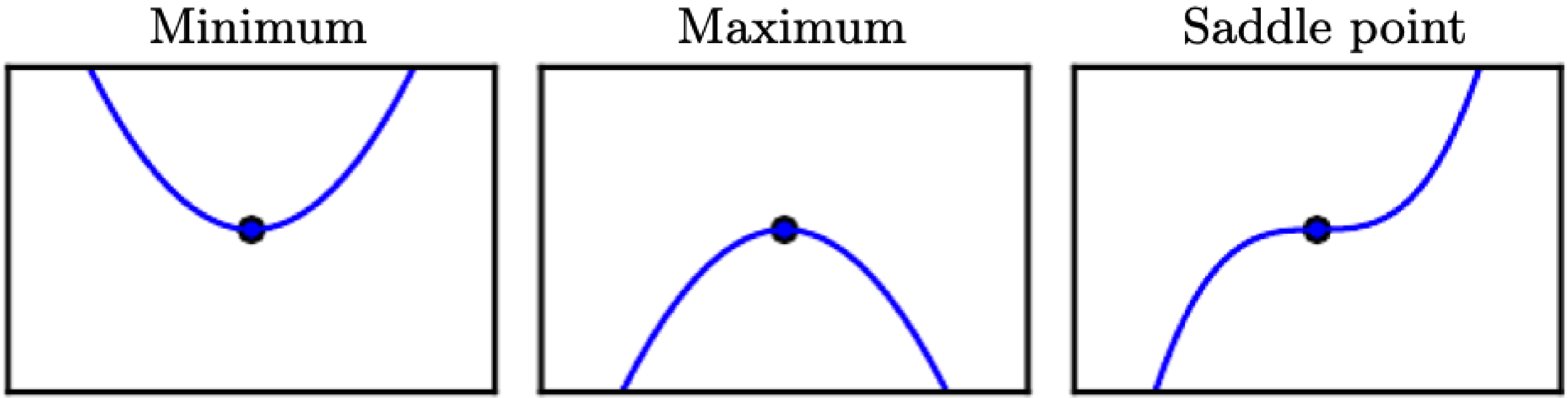
We update x_k via

$$x_{k+1} = x_k - \epsilon f'(x_k)$$

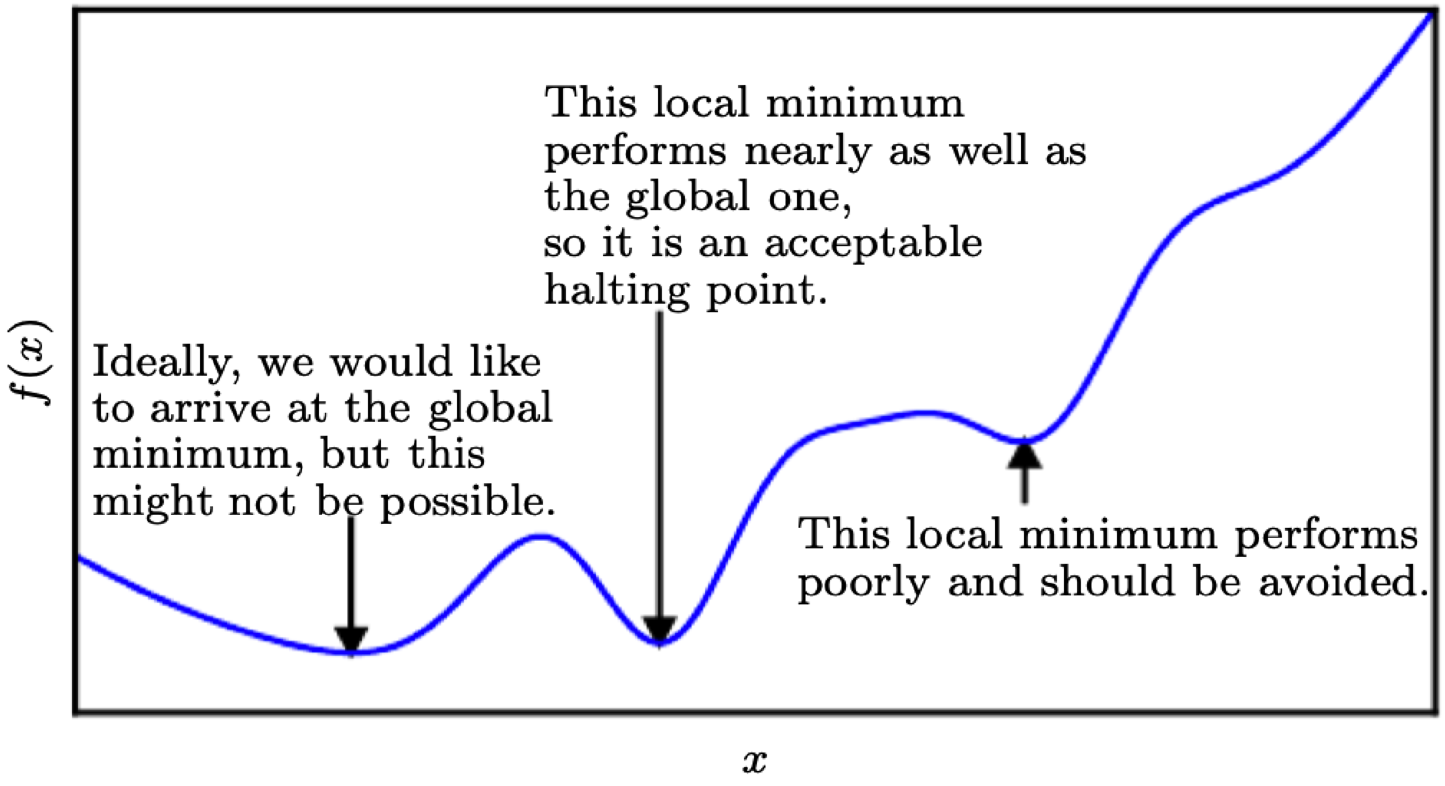


Critical Points

Points where ${}^{\mathrm{d}f(x)}/_{\mathrm{d}x} = 0$ are critical points



Optimization in deep learning



Multivariate functions

We are interested in functions

$$f : \mathbb{R}^n \to \mathbb{R}$$

Then, the gradient generalizes the notion of derivative

$$\nabla_x f(x)_i = \frac{\partial}{\partial x_i} f(x)$$

Multivariate functions

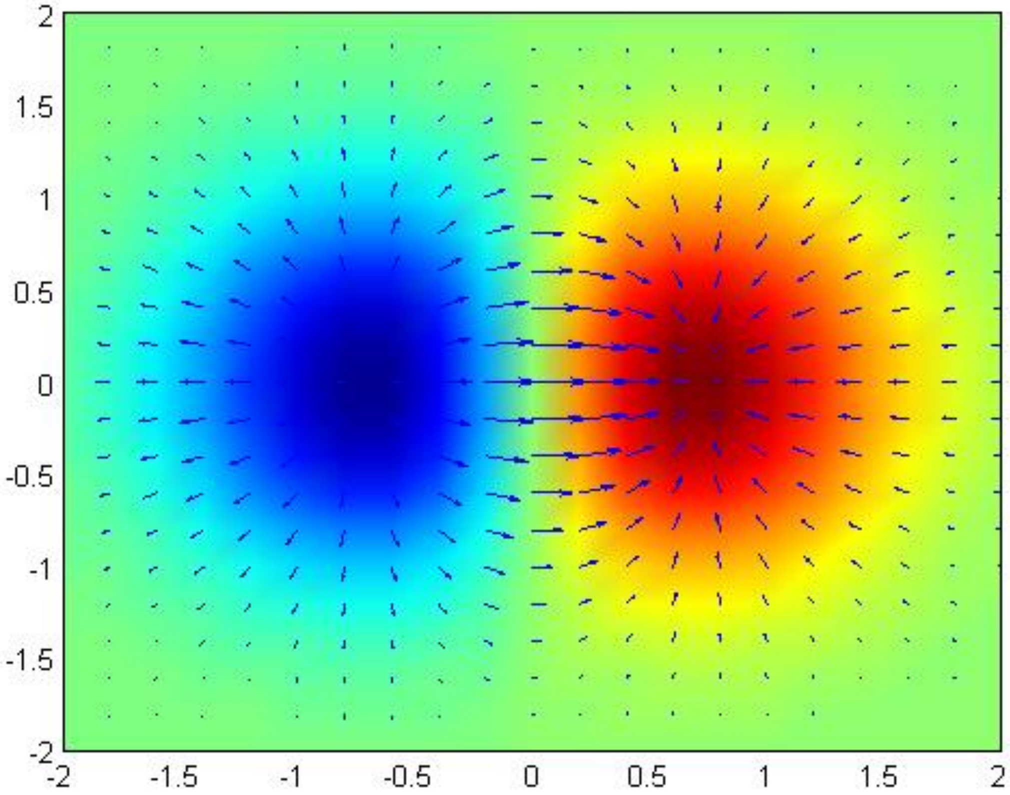
Functions of multiple arguments

$$f: \mathbb{R}^n \rightarrow \mathbb{R}$$

The gradient

$$\nabla_x f(x) = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n}\right)^T$$

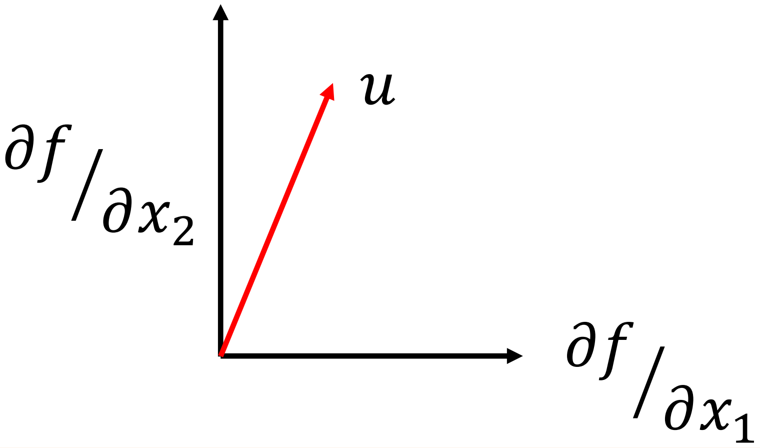
$x \exp(-x^2 - y^2)$



The Method of Steepest Descent

The directional derivative

$$\frac{\partial}{\partial \alpha} f(x + \alpha u) \longrightarrow_{\alpha=0} \; u^T \nabla_x f(x)$$



Where f decreases the fastest?

$$\min_{u, u^T u = 1} \; u^T \nabla_x f(x)$$

$\cos \theta$

Thus

$$x_{k+1} = x_k - \epsilon \nabla_{x_k} f(x)$$

learning rate

Steepest descent

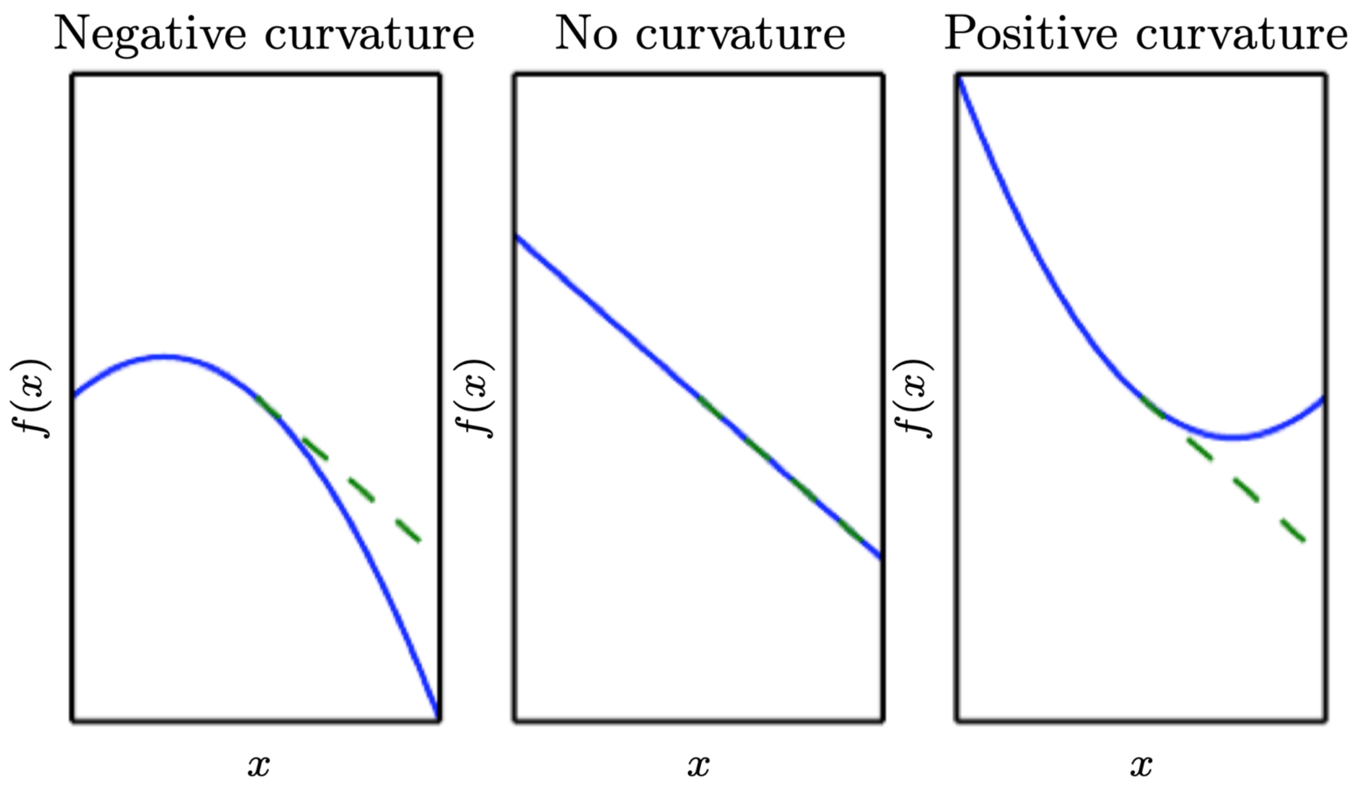
The solution is given by

$$x' = x - \epsilon \nabla_x f(x)$$

learning rate

Second Derivatives

The second derivative $f''(x) = {}^{\mathrm{d}^2 f}\!/_{\mathrm{d}x^2}$ measures the curvature



The Hessian Matrix

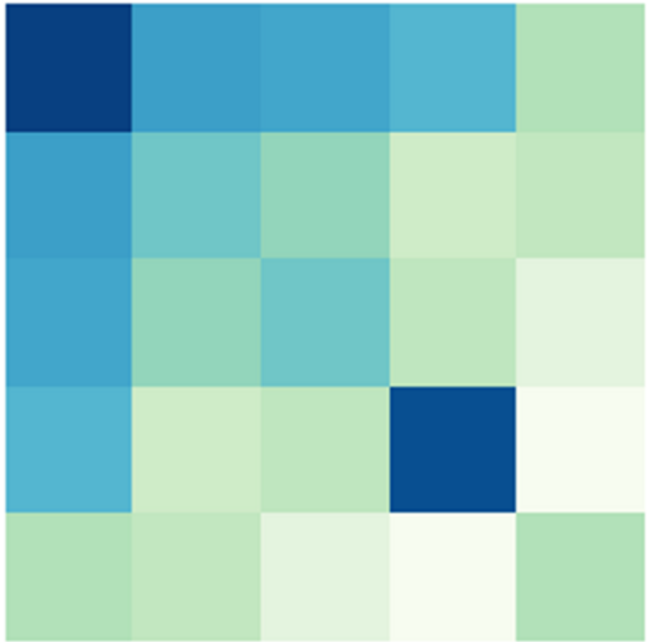
Given a function

$$f : \mathbb{R}^m \to \mathbb{R}$$

Matrix of second partial derivatives

$$\underset{\text{Hessian}}{H_{ij}} = \frac{\partial^2}{\partial x_i \partial x_j} f(x)$$

Hessian is the Jacobian of gradient



Taylor Expansion

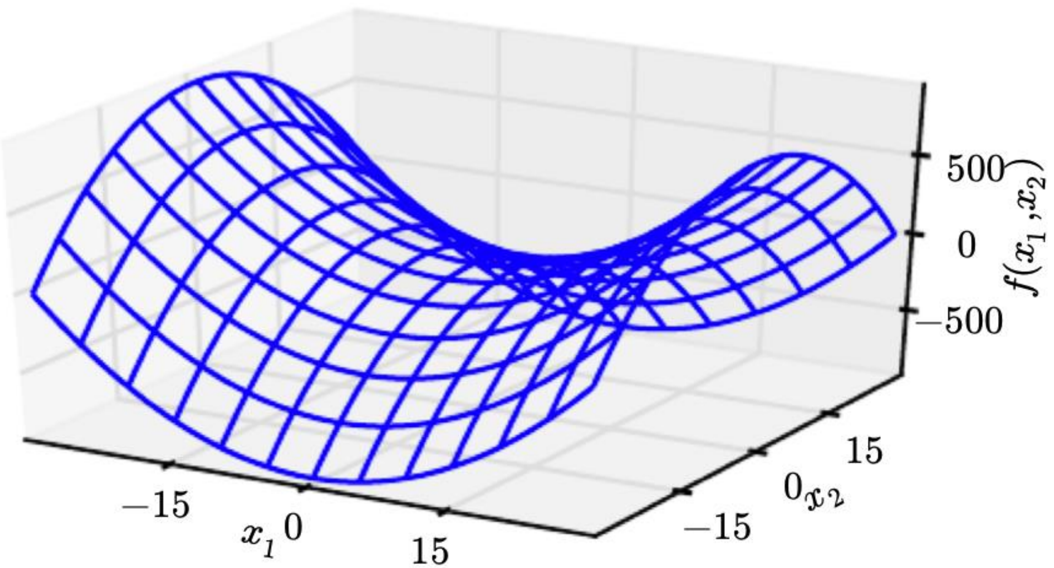
Given a univariate function $f: \mathbb{R} \to \mathbb{R}$

$$f(x) \approx f\left(x^{(0)}\right) + \left(x - x^{(0)}\right) f'\left(x^{(0)}\right) + \frac{1}{2}\left(x - x^{(0)}\right)^2 f''\left(x^{(0)}\right)$$

For multivariate functions $f: \mathbb{R}^n \to \mathbb{R}$

$$f(x) \approx f\left(x^{(0)}\right) + \left(x - x^{(0)}\right)^T \underbrace{g}_{\text{gradient}} + \frac{1}{2}\left(x - x^{(0)}\right)^T \underbrace{H}_{\text{Hessian}} \left(x - x^{(0)}\right)$$

Evaluating critical points



Minima Classification

Univariate $f'(x) = 0$

local minimum: $f''(x) > 0$

local maximum: $f''(x) < 0$

Multivariate $\nabla_x f(x) = 0$

local minimum: H is PD

local maximum: H is ND

Condition Number

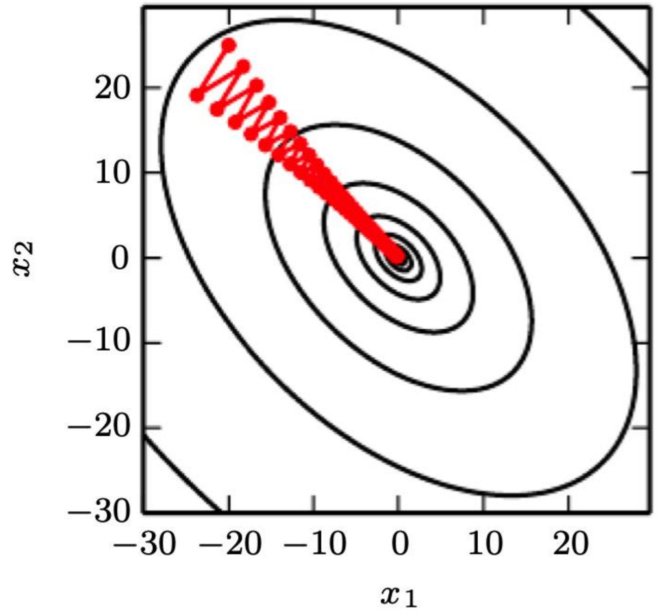
Given $A \in \mathbb{R}^{m\times n}$

$$\max_{i,j}\left|\frac{\lambda_i}{\lambda_j}\right|$$

condition number

ill-conditioned Hessian $\Longrightarrow$

gradient descent performs poorly



Newton's Method

Recall that

$$f(x) \approx f\left(x^{(0)}\right) + \left(x - x^{(0)}\right)^T g + \frac{1}{2}\left(x - x^{(0)}\right)^T H\left(x - x^{(0)}\right)$$

The critical point of the approximation is given by

$$x^* = x^{(0)} - H^{-1} g := x^{(0)} - H\left(x^{(0)}\right)^{-1} \nabla_x f\left(x^{(0)}\right)$$

second-order optimization

Constrained Optimization

Consider an equality constraint

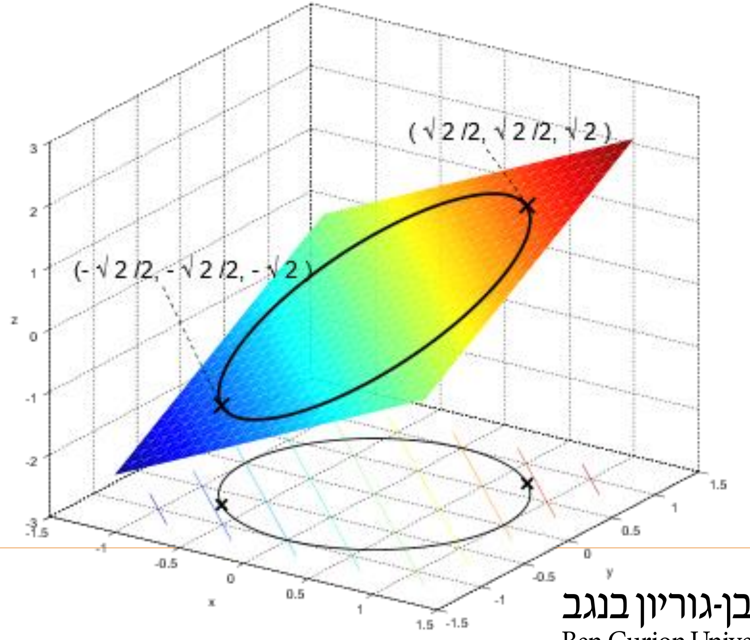
$$g(x) = 0$$

The generalized Lagrangian

$$\mathcal{L}(x, \lambda) = f(x) + \lambda g(x)$$

The unconstrained optimization

$$\min_{x} \max_{\lambda} \mathcal{L}(x, \lambda)$$



Example: Linear Least Squares

Find x that minimizes

$$f(x) = \frac{1}{2}|Ax - b|_2^2$$

The gradient

$$\nabla_x f(x) = A^T Ax - A^T b$$

Algorithm 4.1 An algorithm to minimize $f(\boldsymbol{x}) = \frac{1}{2}||\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}||_2^2$ with respect to $\boldsymbol{x}$ using gradient descent, starting from an arbitrary value of $\boldsymbol{x}$.

Set the step size (ϵ) and tolerance (δ) to small, positive numbers.
while $||\boldsymbol{A}^\top \boldsymbol{A}\boldsymbol{x} - \boldsymbol{A}^\top \boldsymbol{b}||_2 > \delta$ **do**
  $\boldsymbol{x} \leftarrow \boldsymbol{x} - \epsilon\left(\boldsymbol{A}^\top \boldsymbol{A}\boldsymbol{x} - \boldsymbol{A}^\top \boldsymbol{b}\right)$
end while

Example: Linear Least Squares

Stochastic Gradient Decsent (SGD)

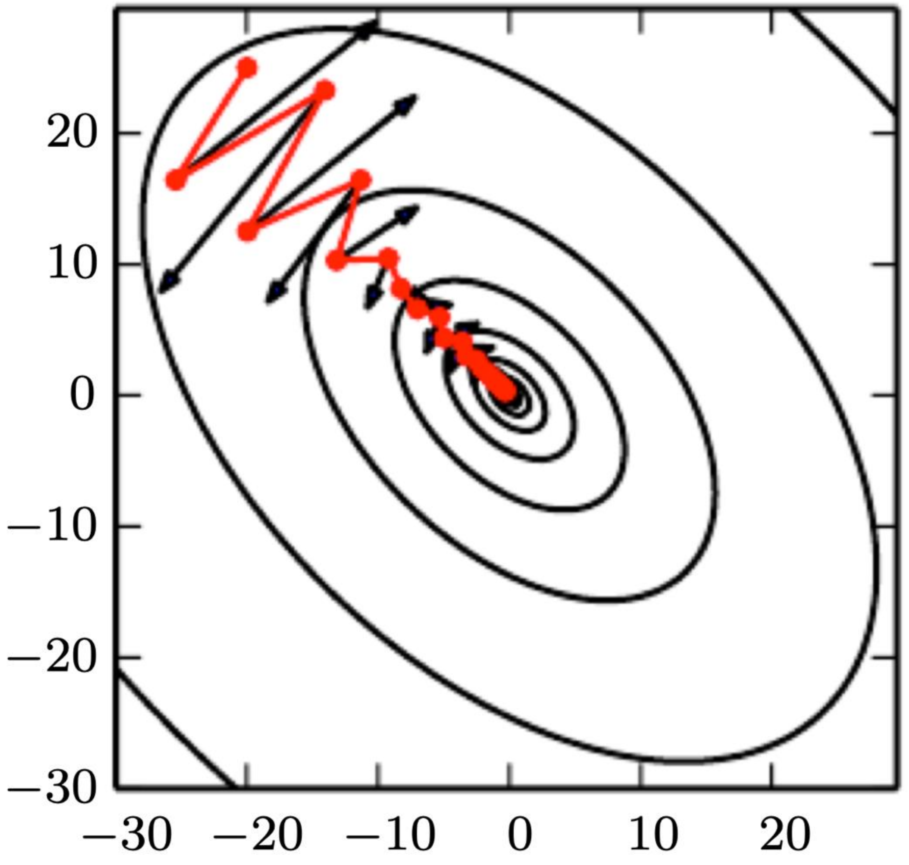
Algorithm 8.1 Stochastic gradient descent (SGD) update at training iteration k

Require: Learning rate ϵ_k.
Require: Initial parameter $\boldsymbol{\theta}$
  while stopping criterion not met **do**
    Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.
    Compute gradient estimate: $\hat{\boldsymbol{g}} \leftarrow +\frac{1}{m}\nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
    Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon \hat{\boldsymbol{g}}$
  end while

Momentum

The velocity is an exponentially decaying average of the gradient

$$v \leftarrow \alpha v - \epsilon \nabla_\theta \mathcal{L}(X^{(i)}, \lambda)$$
$$\theta \leftarrow \theta + v$$



Code Demo